

Architektury systemów komputerowych

Lista zadań nr 11

Na zajęcia 13 i 26 maja 2026

Przydatne informacje można znaleźć w [1, §6.1], [1, §6.2], [1, §6.3] i [1, §6.4].

UWAGA! W trakcie prezentacji rozwiązań należy zdefiniować i wyjaśnić wszystkie pojęcia, które są potrzebne do zrozumienia i rozwiązania zadania.

Zadanie 1. Rozważmy dysk o następujących parametrach: jeden plater; jedna głowica; 576 tysięcy ścieżek na powierzchnię; 32000 sektorów na ścieżkę; 7200 obrotów na minutę; czas wyszukiwania: 1ms na przeskoczenie o 24 tysięcy ścieżek. Odpowiedz na następujące pytania:

1. Jaki jest średni czas wyszukiwania (ang. *seek time*)?
2. Jaki jest średni czas opóźnienia obrotowego (ang. *rotational latency*)?
3. Jaki jest czas transferu (ang. *transfer time*) sektora?
4. Jaki jest całkowity średni czas obsługi żądania?

Zadanie 2. Rozważmy napęd dyskiety 3½ o następujących parametrach: 300 obrotów na minutę, 512 bajtów na sektor, 18 sektorów na ścieżkę, 80 ścieżek na powierzchnię. Dysk sygnalizuje dostępność danych zgłaszając przerwanie na każdy przeczytany bajt, a wykonanie procedury przerwania zajmuje 16μs. Następnie do systemu dodajemy kontroler DMA, więc przerwanie będzie generowane tylko raz po wczytaniu sektora do pamięci. Ile czasu zostanie procesorowi na pozostałe czynności w trakcie czytania n sektorów (a) bez użycia oraz (b) przy użyciu kontrolera DMA? Należy zignorować czas wyszukiwania ścieżki i sektora. W jaki sposób dodanie DMA do systemu poprawiło jego wydajność?

Wskazówka: Więcej informacji na temat przerwań można znaleźć w [1, §8.1.2].

Zadanie 3. Rozważamy umiarkowany nowoczesny procesor x86-64 (np. i7-6700), o częstotliwości taktowania 2.5GHz, z trzema poziomami pamięci podręcznej. Niech funkcja $A(k)$ wyznacza czas, w cyklach procesora, w którym pamięć k -tego poziomu odpowiada na pytanie czy przechowuje dany blok pamięci. Niech funkcja $H(k)$ wyznacza prawdopodobieństwo z jakim blok znajduje się na k -tym poziomie hierarchii pamięci, pod warunkiem, że nie znajduje się na wcześniejszych poziomach. Dla rozważanego systemu mamy: $A(L1) = 4$, $A(L2) = 12$, $A(L3) = 40$, $A(DRAM) = 200$; $H(L1) = 0.9$, $H(L2) = 0.95$, $H(L3) = 0.98$, $H(DRAM) = 1.0$. Jaki jest (a) średni oraz (b) pesymistyczny czas dostępu do pamięci w nanosekundach?

Pamiętaj! Dostęp do pamięci na poziomie $k + 1$ zachodzi tylko wtedy, gdy chybiłmy w pamięć na poziomie k .

Zadanie 4. Załóżmy, że dostęp do pamięci głównej trwa 70ns, a dostępy do pamięci stanowią 36% wszystkich instrukcji. Rozważmy system z pamięcią podręczną o następującej strukturze: L1 – 2 KiB, współczynnik chybień 8.0%, czas dostępu 0.66ns (1 cykl procesora); L2 – 1 MiB, współczynnik chybień 0.5%, czas dostępu 5.62ns. Procesor charakteryzuje się współczynnikiem CPI (ang. *cycles per instruction*) równym 1.0, jeśli pominiemy instrukcje robiące dostępy do pamięci danych. Odpowiedz na poniższe pytania:

- Jaki jest średni czas dostępu do pamięci dla procesora tylko z pamięcią podręczną L1, a jaki dla procesora z L1 i L2?
- Procesor wykonuje wszystkie instrukcje, łącznie z dostęпами do pamięci danych. Oblicz jego CPI kiedy posiada tylko pamięć podręczną L1, oraz gdy posiada L1 i L2.

Uwaga: Zakładamy, że wszystkie instrukcje wykonywane przez program są w pamięci podręcznej L1i.

Zadanie 5. Rozważmy pamięć podręczną, początkowo pustą, która jest w stanie pomieścić 4 bloki danych. Pamięć ta jest w pełni asocjacyjna, co oznacza, że każdy blok danych może być umieszczony w dowolnym z tych 4 miejsc. Dodatkowo, polityką wymiany bloków jest *LRU* (*Least Recently Used*), czyli w momencie gdy musimy usunąć blok danych z pamięci podręcznej, na ofiarę zostaje wybrany ten blok, do którego dostęp odbywał się najdawniej.

Zasymuluj zawartość pamięci podręcznej w przypadku, gdy procesor wymaga kolejno dostępu do bloków:

1, 2, 3, 1, 4, 4, 3, 7, 4, 2, 3, 8, 1, 3, 4, 3.

Które z tych dostępu spowodują trafienia (ang. *hit*), a które chybieńia (ang. *miss*)? Jakiego typu będą to chybieńia (*cold*, *capacity*, *conflict*)? Jak zmieniłaby się sytuacja, gdyby pamięć podręczna była w stanie pomieścić 5 bloków danych?

Zadanie 6. Rozważmy stertę (ang. *heap*) o rozmiarze 2^{20} bajtów. Przyjmijmy, że będziemy alokować (więc również i zwalniać) w niej tylko bloki o dwóch możliwych rozmiarach: 16 bajtów oraz 1024 bajty. Przyjmijmy również, że alokator, w momencie gdy potrzebuje znaleźć miejsce na nowy blok danych, wybiera zawsze pierwsze wolne miejsce, w którym ten blok się mieści.

Stwórz taką sekwencję zapytań (alokacji i zwolnień bloków pamięci), że w momencie, gdy konieczne będzie powiększenie sterty, to jej zapełnienie wartościami danymi (ang. *payload*) będzie (a) możliwie największe, oraz taką sekwencję, że zapełnienie będzie (b) możliwie najmniejsze.

Rozważ przypadki (i) gdy blok pamięci nie wymaga żadnych metadanych (ani nagłówka, ani stopki) oraz (ii) gdy każdy blok potrzebuje nagłówka o rozmiarze 8 bajtów.

Zadanie 7. Rozwiąż poprzednie zadanie w sytuacji, gdy sekwencja nigdy nie będzie zawierała operacji zwolnienia bloku o rozmiarze 16 bajtów.

Rozważmy tę sytuację w wariacie (i). Czy jesteś w stanie zaproponować sposób przydzielania miejsca w pamięci, w którym rozszerzenie sterty nastąpi tylko w momencie, gdy sumaryczna ilość wolnego miejsca będzie zbyt mała?

Literatura

- [1] „*Computer Systems: A Programmer’s Perspective*”
Randal E. Bryant, David R. O’Hallaron; Pearson; 3rd edition, 2016
- [2] „*What Every Programmer Should Know About Memory*”¹
Ulrich Drepper, November 21, 2007
- [3] „*Memory Systems: Cache, DRAM, Disk*”
Bruce Jacob, Spencer W. Ng, David T. Wang; Morgan Kaufmann, 2008

¹<https://www.akkadia.org/drepper/cpumemory.pdf>